Python Scripting

(https://github.com/learn-co-curriculum/python-p3-python-scripting) (https://github.com/learn-co-curriculum/python-p3-python-scripting/issues/new)

Learning Goals

- Learn about scripting.
- Write a script in python.

Key Vocab

- **Module**: a file containing Python definitions and statements. A module's functions, classes, and global variables can be accessed by other modules.
- **Package**: a collection of modules that can be accessed as a group using the package name.
- `import` : the Python keyword used to access data from other packages and modules inside of the current module.
- **PyPI**: the **Py**thon **P**ackage **I**ndex. A repository of Python packages that can be downloaded and made available to your application.
- `pip` : the command line tool used to download packages from PyPI. `pip` is installed on your computer automatically when you download Python.
- **Virtual Environment**: a collection of modules, packages, and scripts that can be activated or deactivated at any time.
- **Pipenv**: a virtual environment tool that uses `pip` to manage the modules, packages, and scripts that you intend to use in your application.

Introduction

The word script in the context of programming refers to a code file containing a program which usually executes some tasks that need to be automated. Scripts are meant to be directly executed by the users.

How are scripts different than the modules we write in python?

Code that is meant to be imported and used in another program is a module. Script are meant to be run directly.

Shebang

The shebang at the beginning of a script allows us to run our python script as an executable without typing `python` before the file name.

```
#!/usr/bin/env python3
```

If we add this code in the beginning of the script the user can run the script using `./script.py` instead of `python script.py`

---

if name == "main":

```
if __name__ == "__main__":
    # do things
```

You may have seen this syntax before when working with python or looking at python examples. This tells python that you only want to run the the code if the program is executed as a standalone file. If we import the script into another program the code in the if statement will not run because the `__name__` variable will not be equal `"__main__"` .

Accepting arguments from command line

Using the sys module we can accept inputs from the user from the command line. Lets use the `sys.argv` list which allows us to get input from the user and print it.

Lets look at the following script

```
#!/usr/bin/env python3
import sys
# The user input starts at index 1 in the sys.argv list.
name = sys.argv[1]
if __name__ == "__main__":
    print(f"The name is {name} ")
```

We can run this script in the terminal using the following command

```
$ ./bin/script.py Steve
The name is Steve
```

Using os.system to Run a shell command

We can also run shell commands using the `os` module Lets try to use the `ls -l` shell command in our script.

```
#!/usr/bin/env python3
import os
if __name__ == "__main__":
    os.system('ls -l')
```

The script will give us a list of files that are in the current directory.

```
$ ./bin/script.py
-rw-r--r--  1 user  staff  1810 Jul 28 19:23 CONTRIBUTING.md
-rw-r--r--  1 user  staff  1346 Jul 28 19:23 LICENSE.md
-rw-r--r--@ 1 user  staff  3653 Sep 27 22:30 README.md
drwxr-xr-x  3 user  staff    96 Sep 27 21:53 bin
```

Bash Scripts

A Bash script is a text file which can contains a series of commands. These commands can help us automate tasks that would require the developer to type multiple commands in the terminal. Any task you can run in the terminal can be put in a Bash script. All you need to do is add the commands sequentially in the Bash file. For Bash scripts the convention is to use the `.sh` file extension.

Lets take a look at a Bash script.

```
#!/bin/bash
echo "Files in this directory"
ls -a
```

Before running the script you may need to give it executable permission. You can change the permissions by running `chmod +x script.sh`. Now we can run the script using the following command.

```
./script.sh
```

The output should return a message and the output of the `ls` command will output the files of the directory the script is being run in.

```
Files in this directory
.               .canvas         .github         LICENSE.md      Pipfile.lock    bin
..              .git            CONTRIBUTING.md Pipfile         README.md       script.sh
```

Conclusion

We have covered some introductory concepts in scripting. You have learned how to accept inputs from the users and run system commands in a script. These tools can help us become much more productive by automating tasks and procedures.

Resources

- **Python (https://docs.python.org/3/)**
- **Python os.system (https://docs.python.org/3/library/os.html#os.system)**

- **Python sys (https://docs.python.org/3/library/sys.html)**